Experiment 13: Artificial Neural Network

Aim:

To implement an Artificial Neural Network that predicts whether a patient has a disease from medical records, and to train and test the model.

Algorithm:

1. Prepare the scaled input features (age, blood pressure, cholesterol, glucose) and the target label.
2. Initialize weights randomly and biases to zero for the input-to-hidden and hidden-to-output layers.
3. Forward pass: compute hidden and output activations using the sigmoid function.
4. Calculate the error between the predicted and the actual output.
5. Backpropagate the error and update all weights and biases using gradient descent.
6. Repeat for a number of epochs, then test the trained network on new patient data.

Code:

```
import numpy as np

# Features: [Age (scaled), BloodPressure (scaled), Cholesterol (scaled), Glucose (scaled)]
X = np.array([
    [0.2, 0.3, 0.2, 0.3], [0.8, 0.9, 0.8, 0.9], [0.3, 0.4, 0.3, 0.2],
    [0.9, 0.8, 0.9, 0.8], [0.1, 0.2, 0.3, 0.2], [0.7, 0.8, 0.7, 0.9],
    [0.4, 0.3, 0.4, 0.3], [0.8, 0.7, 0.9, 0.8], [0.2, 0.2, 0.1, 0.3],
    [0.9, 0.9, 0.8, 0.7], [0.3, 0.3, 0.2, 0.4], [0.7, 0.9, 0.8, 0.8],
])
y = np.array([[0], [1], [0], [1], [0], [1], [0], [1], [0], [1], [0], [1]])  # 1 = Disease

np.random.seed(1)
W1 = np.random.randn(4, 6) * 0.5
b1 = np.zeros((1, 6))
W2 = np.random.randn(6, 1) * 0.5
b2 = np.zeros((1, 1))

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

lr = 0.5
for epoch in range(2001):
    h = sigmoid(X @ W1 + b1)          # forward pass
    out = sigmoid(h @ W2 + b2)
    loss = np.mean((y - out) ** 2)
    d_out = (out - y) * out * (1 - out)   # backpropagation
    d_h = (d_out @ W2.T) * h * (1 - h)
    W2 -= lr * h.T @ d_out
    b2 -= lr * d_out.sum(axis=0, keepdims=True)
    W1 -= lr * X.T @ d_h
    b1 -= lr * d_h.sum(axis=0, keepdims=True)
    if epoch % 500 == 0:
        print(f"Epoch {epoch}, Loss: {loss:.4f}")

def predict(x):
    return sigmoid(sigmoid(np.array(x) @ W1 + b1) @ W2 + b2)

pred = (predict(X) > 0.5).astype(int)
print("Training Accuracy:", np.mean(pred == y))

test = [[0.85, 0.8, 0.9, 0.85], [0.2, 0.25, 0.2, 0.3]]
for t in test:
    p = predict([t])[0][0]
    print("Patient", t, "-> Probability:", round(float(p), 3),
          "-> Disease" if p > 0.5 else "-> No Disease")
```

Sample Output:

Output

```
Epoch 0, Loss: 0.2422
Epoch 500, Loss: 0.0006
Epoch 1000, Loss: 0.0002
Epoch 1500, Loss: 0.0001
Epoch 2000, Loss: 0.0001
Training Accuracy: 1.0
Patient [0.85, 0.8, 0.9, 0.85] -> Probability: 0.993 -> Disease
Patient [0.2, 0.25, 0.2, 0.3] -> Probability: 0.002 -> No Disease
```

Result:

The loss fell from 0.2422 to almost zero over 2000 epochs and the network reached 100% training accuracy. It correctly predicted Disease for the high-risk patient and No Disease for the low-risk patient.